

Gazebo Simülasyonları için Kapsamlı XML Düğüm Referansı

SDF ve URDF Öğelerini Anlama Kılavuzu

I. Giriş: SDF ve URDF'nin Temel Yapı Taşları

Gazebo simülasyon ortamı, robotik sistemlerin ve çevrelerinin davranışını tanımlamak için güçlü bir XML tabanlı yapı kullanır. Eğitimlerde (tutorial) karşılaşılan XML "düğümleri" (nodes) veya daha doğru bir terimle "öğeleri" (elements), bu simülasyonların temel yapı taşlarıdır. Bu düğümlerin anlaşılması, iki farklı ancak ilişkili formatın, SDF (Simulation Description Format) ve URDF (Unified Robot Description Format), rollerini ve etkileşimlerini kavramayı gerektirir.

A. XML Düğümlerinin İkili Doğası: SDF vs. URDF

Gazebo ekosisteminde temel olarak iki XML formatı bulunur ve bu formatların farklı amaçları vardır:

- SDF (Simulation Description Format):** Bu, Gazebo'nun yerel, kapsamlı formatıdır. Bir SDF dosyası (genellikle `.world` veya `.sdf` uzantılı), bir simülasyon dünyasının tüm yönlerini tanımlayabilir:
 - Fizik motoru ayarları (örn. yerçekimi, zaman adımı).
 - Işık kaynakları (örn. güneş).
 - Robotik olmayan modeller (örn. binalar, mobilyalar).
 - Sensörler, eklentiler ve aktörler (animasyonlu karakterler). Kısacası SDF, simülasyonun kendisidir.
- URDF (Unified Robot Description Format):** Bu, öncelikle ROS (Robot Operating System) ekosisteminin standart formatıdır. Bir URDF dosyası, bir robotun *kinematik* ve *görsel* tanımına odaklanır:
 - Robotun parçalarını (linkler) ve bu parçaların nasıl bağlandığını (joint'ler) tanımlar.
 - Fiziksel özellikleri (kütle, atalet) tanımlayabilir.
 - Ancak URDF, sensörleri, ışıkları, kapalı kinematik zincirleri (örn. dört çubuk mekanizması) veya eklentileri yerel olarak **desteklemez**.

B. Köprü Olarak `<gazebo>` Etiketi

ROS tabanlı bir robotun Gazebo'da simüle edilmesi gerektiğinde, bu iki format arasındaki boşluk bir sorun yaratır. Gazebo bu sorunu, bir URDF dosyasını yüklerken onu dahili olarak bir SDF modeline dönüştürerek çözer.

Bu dönüşüm sürecinde kritik bir rol oynayan özel bir etiket vardır: `<gazebo>`. Bu etiket, bir URDF dosyası içinde bir "XML kaçış ambarı" (escape hatch) veya "doğrudan geçiş" (pass-through) mekanizması olarak işlev görür. Bir URDF ayrıştırıcısı bu etiketi görmezden gelir, ancak Gazebo'nun URDF'den-SDF'ye dönüştürücüsü bu etiketin *içeriğini* okur ve doğrudan oluşturulan SDF modeline kopyalar.

Bu mekanizma, URDF'nin eksik özelliklerini (sensörler, eklentiler ve SDF'ye özgü fizik ayarları) URDF dosyasına "enjekte etmek" için kullanılır.

C. Terminoloji: Düğüm (Node), Öğe (Element) ve Öznitelik (Attribute)

Bu rapor boyunca, XML yapısını net bir şekilde tanımlamak için standart terminoloji kullanılacaktır:

- **Öge (Element) / Düğüm (Node):** Açılış ve kapanış etiketleri tarafından tanımlanan yapısal birimdir (örn. `<link>...</link>`). Kullanıcının "node" (düğüm) talebi bu kavrama karşılık gelmektedir.
- **Öznitelik (Attribute):** Bir ögenin açılış etiketi içinde yer alan ve o ögenin bir özelliğini tanımlayan bir ad-değer çiftidir (örn. `<joint type="revolute">` ifadesindeki `type="revolute"`).

Eğitimlerde görülen ve kafa karıştırıcı olabilen sözdizimi çeşitliliği (örneğin, bazen `<sensor>` görmek, bazen de `<gazebo reference="..."><sensor>...</gazebo>` görmek) bir tutarsızlık değildir. Bu, tam olarak yukarıda açıklanan SDF ve URDF ikiliğinin doğrudan bir semptomudur. Bir geliştiricinin yapması gereken ilk şey, düzenlediği dosyanın saf bir SDF/dünya dosyası mı (SDF sözdizimini kullanır) yoksa ROS entegrasyonu için bir URDF/XACRO dosyası mı (SDF özelliklerini enjekte etmek için `<gazebo>` etiketini kullanır) olduğunu belirlemektir. Bu ikilik, Gazebo/ROS entegrasyonunda hata ayıklamanın merkezinde yer alır.

II. Dünya (World) ve Simülasyon Yapılandırma Dükümüleri (SDF)

Bu bölüm, `.world` dosyalarında bulunan ve simülasyonun genel kurallarını, fiziğini ve ortamını belirleyen en üst düzey SDF öğelerine odaklanmaktadır.

A. Fizik Motorunun Ayarlanması: `<physics>` Dükümü

Bu düküm, simülasyonun fiziksel gerçekliğini ve performansını yöneten temel parametreleri tanımlar.

Amacı: Simülasyonun fizik motorunu (varsayılan olarak `ode` - Open Dynamics Engine), zaman adımlarını ve çözücü özelliklerini yapılandırır.

Kod Örneği: Bu örnek, farklı eğitimlerden alınan temel parametreleri birleştirir.

```
<physics type="ode" name="ode_default_profile">
  <max_step_size>0.001</max_step_size>
  <real_time_update_rate>1000</real_time_update_rate>
  <ode>
    <solver>
      <type>quick</type>
      <iters>70</iters>
      <sor>1.3</sor>
      <use_dynamic_moi_rescaling>>false</use_dynamic_moi_rescaling>
    </solver>
    <constraints>
      <cfm>0.0</cfm>
      <erp>0.2</erp>
      <contact_max_correcting_vel>0.1</contact_max_correcting_vel>
      <contact_surface_layer>0.0001</contact_surface_layer>
    </constraints>
  </ode>
</physics>
```

Alt Dükümlerin Analizi:

- `<max_step_size>` : Bir simülasyon adımında ilerlenecek simülasyon zamanıdır (saniye cinsinden). Varsayılanı 0.001'dir. Değer küçüldükçe doğruluk artar ancak hesaplama yükü de artar.
- `<real_time_update_rate>` : Simülasyonun saniyede kaç kez güncellenmeye çalışacağını belirler (Hz cinsinden). Varsayılanı 1000'dir. Eğer 0 (sıfır) olarak ayarlanırsa, simülasyon donanımın izin verdiği en yüksek hızda (mümkün olduğunca hızlı) çalışır.
- `<ode>` : Open Dynamics Engine (ODE) fizik motoruna özgü ayarları içeren sarmalayıcı.
- `<solver>` : Fiziksel kısıtlamaların (joint'ler ve temaslar) nasıl çözüleceğini tanımlar. `<iters>`

(iterasyonlar) özneliği, özellikle `type="quick"` çözücü için önemlidir ve joint kararlılığını büyük ölçüde etkiler.

- `<constraints>` : Kısıtlamalar için global parametreleri ayarlar. `<cfm>` (Constraint Force Mixing) ve `<erp>` (Error Reduction Parameter), joint'lerin ve temasların kararlılığını ayarlamak için kullanılan kritik, ancak genellikle anlaşılması zor parametrelerdir.

Simülasyon performansını ayarlarken, `<max_step_size>` ve `<real_time_update_rate>` arasındaki ilişkiyi anlamak hayati önem taşır. Bu iki değerin çarpımı, "gerçek zaman faktörünü" (real-time factor - RTF) belirler. Varsayılan ayar olan $0.001 * 1000 = 1.0$, simülasyonun gerçek zamanlı (1x hız) çalışmasını hedefler. Eğer bir kullanıcı daha yüksek doğruluk için `<max_step_size>` 'ı 0.0001'e düşürürse (10 kat daha küçük adım), simülasyon 0.1x hızına yavaşlayacaktır. Gerçek zamanlılığı (1.0) korumak için, `<real_time_update_rate>` 'in 10000'e (10 kat daha fazla güncelleme) çıkarılması gerekir ki bu da CPU yükünü önemli ölçüde artırır. Bu, simülasyon doğruluğu ve performans arasındaki temel değiş-tokuştur.

B. Simülasyon Eklentileri: `<plugin>` ve `<gui>` Düşümleri

Eklentiler (Plugins), simülasyonun davranışını veya arayüzünü değiştiren, C++ ile yazılmış ve derlenmiş paylaşımlı kütüphanelerdir (`.so` dosyaları). Gazebo'nun temel mimarisi, iki ana sürece ayrılmıştır: `gzserver` (fizik sunucusu) ve `gzclient` (grafik arayüzü). Bu bölünmüş yapı, eklentilerin nasıl yüklendiğini de belirler.

Tür 1: Dünya (World) Eklentisi

- **Bağlam:** Bu eklentiler doğrudan `<world>` öğesinin altına yerleştirilir. Fizik sunucusu (`gzserver`) üzerinde çalışırlar ve simülasyonun kendisini (örn. fizik kuvvetleri uygulamak, model oluşturmak/silmek) etkilerler. Arayüzsüz (headless) modda çalışırken bile etkindirler.
- **Kod Örneği:**

```
<sdf version="1.4">
  <world name="default">
    <plugin name="hello_world" filename="libhello_world.so"/>
  </world>
</sdf>
```

Tür 2: GUI Eklentisi

- **Bağlam:** Bu eklentiler `<gui>` öğesinin altına yerleştirilir. Gazebo istemcisi (`gzclient`) üzerinde çalışırlar. Fizik dünyasını *etkilemezler*, yalnızca kullanıcı arayüzüne yeni düğmeler, pencereler veya görselleştirmeler eklerler.
- **Kod Örneği:**

```
<sdf version="1.5">
  <world name="default">
    ...
    <gui>
      <plugin name="sample" filename="libgui_example_spawn_widget.so"/>
    </gui>
    ...
  </world>
</sdf>
```

Yeni başlayanlar için yaygın bir hata, bir GUI eklentisini `<world>` bloğuna (veya tersi) yerleştirmektir. Bu

durumda eklenti sessizce yüklenemez, çünkü yanlış süreçte (sunucu yerine istemci veya tersi) başlatılmaya çalışılır.

III. Robot Modeli Anatomisi: Linkler ve Joint'ler (SDF & URDF)

Bir robot modeli, rijit gövdelerin (linkler) birbiriyle kısıtlı hareketlerle (joint'ler) bağlanmasıyla oluşan bir koleksiyondur.

A. Modelin Temel Bileşeni: `<link>` Düğümü

- **Amacı:** Bir modelin tek bir rijit (sert) gövdesini temsil eder. Bir link, bir tekerlek, bir robot kolunun bir parçası veya bir kamera gövdesi olabilir. Her linkin fiziksel özellikleri, çarpışma geometrisi ve görsel temsili vardır.
- **Kod Örneği:**

```
<link name="link">
  <pose>0.05 0.05 0.05 0 0 0</pose>
  <inertial>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.000166667</ixx>
      <iyy>0</iyy>
      <izz>0</izz>
      <ixy>0</ixy>
      <iyz>0</iyz>
      <ixz>0</ixz>
    </inertia>
  </inertial>
  <collision name="collision">
    <geometry>
      <box>
        <size>0.1 0.1 0.1</size>
      </box>
    </geometry>
  </collision>
  <visual name="visual">
    <geometry>
      <box>
        <size>0.1 0.1 0.1</size>
      </box>
    </geometry>
    <material>
      <script>Gazebo/WoodPallet</script>
    </material>
  </visual>
</link>
```

Alt Düğümlerin Analizi:

- `<inertial>` : Linkin kütesini (`<mass>`) ve atalet matrisini (`<inertia>`) tanımlayarak fiziksel davranışını belirler.
- `<collision>` : Fizik motoru tarafından çarpışma tespiti için kullanılan geometriyi (`<geometry>`) tanımlar.
- `<visual>` : Simülasyonun grafik arayüzünde görünen geometriyi ve malzemeyi (`<material>`) tanımlar.

`<visual>` ve `<collision>` öğelerinin neden ayrı olduğu, simülasyon optimizasyonunun temel bir

ilkesidir. En iyi uygulama, `<visual>` içinde yüksek çözünürlüklü, estetik açıdan karmaşık bir 3D ağ (`<mesh>`) kullanmak, ancak `<collision>` içinde fizik motorunun hızlı hesaplama yapabilmesi için basit bir geometrik şekil (`<box>`, `<sphere>`, `<cylinder>`) veya düşük poligonlu bir dışbükey kaplama (convex hull) kullanmaktır. Karmaşık ağırları çarpışma tespiti için kullanmak, simülasyon performansını önemli ölçüde düşürür.

B. Linkleri Bağlama: `<joint>` Düğümü (SDF)

- **Amacı:** İki `<link>` öğesini birbirine bağlar ve aralarındaki hareket kısıtlamasını (serbestlik derecesi) tanımlar.
- **Kod Örneği:**

```
<joint name="bar_12_joint" type="revolute">
  <parent>link_1</parent>
  <child>link_2</child>
  <pose>0 0.5 0 0 0 0</pose>
  <axis>
    <xyz>0 0 1</xyz>
  </axis>
  <physics>
    <ode>
      <limit>
        <lower>-1.5707</lower>
        <upper>1.5707</upper>
      </limit>
    </ode>
  </physics>
</joint>
```

Alt Düğümlerin Analizi:

- **type (Öznitelik):** Joint tipini belirler: `revolute` (döner), `prismatic` (kayar), `fixed` (sabit), `ball` (top), `revolute2` (çift eksenli döner).
- **<parent> ve <child> :** Kinematik zinciri tanımlar. `parent`, ebeveyn linki; `child`, çocuk linki belirtir.
- **<axis> :** Hareketin gerçekleştiği eksen (vektör olarak) tanımlar.

Joint'leri bağlarken, özellikle iç içe geçmiş modellerle çalışırken "isim alanı" (namespacing) kritik bir öneme sahiptir. Basit bir modelde `parent` ve `child` adları `link_1` ve `link_2` olabilir. Ancak, `my_robot` adında bir modeliniz varsa ve bu modelin içindeki `wheel_link` 'e bir joint bağlamak istiyorsanız, ebeveyn veya çocuk linkin adı `my_robot::wheel_link` şeklinde, `::` ayırıcısı kullanılarak tam olarak belirtilmelidir. Bu, iç içe geçmiş model linklerine yapılan referansların "kapsama alınması" (scoped) gerektiği anlamına gelir ve yeni başlayanlar için yaygın bir hata kaynağıdır.

C. ROS Entegrasyonu için Aktüatörler: `<transmission>` Düğümü (URDF)

Bu düğüm, Gazebo (SDF) için değil, URDF formatı için spesifiktir ve `gazebo_ros_control` eklentisi (Bölüm V'te detaylandırılmıştır) ile birlikte çalışır.

- **Amacı:** Soyut bir URDF joint'ini, ROS kontrolörlerinin (örn. PID kontrolör) komut gönderebileceği somut bir donanım arayüzüne (hardware interface) bağlayan "yapıştırıcı" görevi görür.
- **Kod Örneği:**

```
<transmission name="tran1">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="joint1">
    <hardwareInterface>EffortJointInterface</hardwareInterface>
  </joint>
  <actuator name="motor1">
    <hardwareInterface>EffortJointInterface</hardwareInterface>
    <mechanicalReduction>1</mechanicalReduction>
  </actuator>
</transmission>
```

Alt Düğümlerin Analizi:

- `<joint name="...">` : Bu ad, aynı URDF dosyasında *başka bir yerde* tanımlanan bir `<joint>` ögesinin `name` özneliliğiyle tam olarak eşleşmelidir.
- `<hardwareInterface>` : Bu, `gazebo_ros_control` eklentisine bu joint'in nasıl kontrol edileceğini söyler. `EffortJointInterface` (efor/tork komutları), `PositionJointInterface` (pozisyon komutları) veya `VelocityJointInterface` (hız komutları) olabilir.

IV. Algılama ve Sensör Düğümleri (SDF & URDF Eklentileri)

Sensörler, simülasyon dünyasından veri toplayan temel bileşenlerdir. SDF, sensörleri yerel olarak desteklerken, URDF bu özellik için `<gazebo>` etiketine güvenir.

A. Sensörün Temel Yapısı: `<sensor>` Düğümü

- **Amacı:** Bir sensör eklentisini barındıran ve genel özelliklerini (isim, tip, güncelleme hızı) tanımlayan bir sarmalayıcıdır. Bir sensör her zaman bir `<link>` ögesinin içine yerleştirilmelidir, çünkü sensörün kendisi fiziksel bir varlığa (linke) bağlıdır.
- **Kod Parçacığı (Genel Yapı):**

```
<sensor name="camera" type="camera">
  <always_on>1</always_on>
  <update_rate>30</update_rate>
  <visualize>true</visualize>
</sensor>
```

B. Sensör Tipleri ve Kod Örnekleri

Aşağıda yaygın sensör tipleri için SDF tanımları yer almaktadır.

Kamera: `<camera>`

```
<sensor name="camera" type="camera">
  <camera>
    <horizontal_fov>1.047</horizontal_fov>
    <image>
      <width>1024</width>
      <height>1024</height>
    </image>
    <clip>
      <near>0.1</near>
      <far>100</far>
    </clip>
  </camera>
```

```
</sensor>
```

Lidar (Lazer): `<ray>`

```
<sensor name="laser" type="ray">
  <ray>
    <scan>
      <horizontal>
        <samples>640</samples>
        <resolution>1</resolution>
        <min_angle>-2.26889</min_angle>
        <max_angle>2.26889</max_angle>
      </horizontal>
    </scan>
    <range>
      <min>0.08</min>
      <max>10</max>
      <resolution>0.01</resolution>
    </range>
  </ray>
</sensor>
```

IMU (Ataletsel Ölçüm Birimi): `<imu>`

```
<sensor name="imu" type="imu">
  <imu>
</imu>
</sensor>
```

Kontak Sensörü: `<contact>`

```
<sensor name='my_contact' type='contact'>
  <contact>
    <collision>box_collision</collision>
  </contact>
</sensor>
```

- **Kritik Not:** Buradaki `<collision>` etiketi, bir geometri tanımlamaz. Bunun yerine, sensörün izleyeceği, ebeveyn linkin `<collision>` öğesinin `name` özniteliğiyle (örn. `<collision name="box_collision">`) eşleşen bir dize içerir.

C. Gerçekçilik Ekleme: `<noise>` **Düğümü**

Gerçek dünyadaki sensörler mükemmel değildir. SDF, simüle edilmiş sensör verilerine istatistiksel gürültü eklemek için `<noise>` bloğunu sağlar.

Kod Örneği (Kamera/Lidar için Basit Gürültü):

```
<noise>
  <type>gaussian</type>
  <mean>0.0</mean>
  <stddev>0.01</stddev>
</noise>
```

Kod Örneği (IMU için Karmaşık Gürültü): IMU'lar, her biri kendi gürültü modeline sahip altı ayrı veri kanalı (3 eksen açısal hız, 3 eksen doğrusal ivme) ürettikleri için daha karmaşık bir gürültü yapısına sahiptirler. Bu hiyerarşik yapı, SDF'nin sensörün veri çıkışının yapısını nasıl yansıttığını gösterir. Bu, her kanal için `bias` (kayma) ve `bias_stddev` (kayma kararlılığı veya "random walk") gibi son derece spesifik, yüksek sadakatli gürültü parametreleri tanımlanmasına olanak tanır.

```
<imu>
  <angular_velocity>
    <x>
      <noise type="gaussian">
        <mean>0.0</mean>
        <stddev>2e-4</stddev>
        <bias_mean>0.0000075</bias_mean>
        <bias_stddev>0.000008</bias_stddev>
      </noise>
    </x>
    <y>...</y>
    <z>...</z>
  </angular_velocity>
  <linear_acceleration>
    <x>
      <noise type="gaussian">
        <mean>0.0</mean>
        <stddev>1.7e-2</stddev>
        <bias_mean>0.1</bias_mean>
        <bias_stddev>0.001</bias_stddev>
      </noise>
    </x>
    <y>...</y>
    <z>...</z>
  </linear_acceleration>
</imu>
```

D. URDF'de Sensör Tanımlama: `<gazebo reference="...">` Düşümü

Daha önce belirtildiği gibi, URDF yerel olarak `<sensor>` öğelerini desteklemez. Bir URDF dosyasına sensör eklemek için, `<gazebo>` etiketi, belirli bir `<link>` 'e referans vererek (örn. `reference="camera_link"`) kullanılır. Bu blok, o linke tam bir SDF sensör tanımını "enjekte eder".

Bu yaklaşım, sensör ve ROS eklentisi arasında kritik bir birleşimi de gösterir. Aşağıdaki örnekte, `<plugin>` öğesi doğrudan `<sensor>` öğesinin içine yerleştirilmiştir. Bu, "Sensör Eklentisi" olarak bilinen özel bir eklenti türüdür. Bu eklentinin (örn. `libgazebo_ros_camera.so`) görevi, ebeveyn `<sensor>` öğesinden (kamera) simüle edilmiş verileri almak ve bu verileri ROS konuları (örn. `<imageTopicName>`) aracılığıyla yayınlamaktır. Bu, simülasyon (SDF) ile ROS (eklenti) arasındaki en sıkı entegrasyon noktasıdır.

Kod Örneği:

```
<gazebo reference="camera_link">
  <sensor type="camera" name="camera1">
    <update_rate>30.0</update_rate>
    <camera name="head">
      <horizontal_fov>1.3962634</horizontal_fov>
      <image>
        <width>800</width>
        <height>800</height>
        <format>R8G8B8</format>
      </image>
      <clip>
```



```

    <near>0.02</near>
    <far>300</far>
  </clip>
</camera>
<plugin name="camera_controller" filename="libgazebo_ros_camera.so">
  <alwaysOn>true</alwaysOn>
  <updateRate>0.0</updateRate>
  <cameraName>rrbot/camera1</cameraName>
  <imageTopicName>image_raw</imageTopicName>
  <cameraInfoTopicName>camera_info</cameraInfoTopicName>
  <frameName>camera_link</frameName>
</plugin>
</sensor>
</gazebo>

```

V. Eklentiler ve ROS Kontrolü (URDF)

Bir robotu simülasyonda sadece görmek değil, aynı zamanda ROS aracılığıyla kontrol etmek de istenir. Bu, `gazebo_ros_control` eklentisi ve Bölüm III.C'de bahsedilen `<transmission>` etiketleri aracılığıyla gerçekleştirilir.

A. Gazebo Eklenti Yöneticisi: `gazebo_ros_control`

- **Amacı:** Bu eklenti, ROS kontrol çerçevesi (`ros_control`) ile Gazebo'nun joint motorları arasında bir arayüz sağlar. URDF'de tanımlanan `<transmission>` etiketlerini okur ve uygun donanım arayüzlerini (örn. Efor, Pozisyon) ROS kontrol yöneticisine kaydeder.
- **Bağlam:** Bu eklenti belirli bir linke *bağlı değildir*. Genellikle URDF dosyasının kök `<robot>` ögesinin sonuna yakın bir yere yerleştirilir.
- **Kod Örneği:**

```

<gazebo>
  <plugin name="gazebo_ros_control" filename="libgazebo_ros_control.so">
    <robotNamespace>/MYROBOT</robotNamespace>
    <controlPeriod>0.001</controlPeriod>
  </plugin>
</gazebo>

```

Alt Düğümlerin Analizi:

- `<robotNamespace>` : Bu eklenti tarafından yüklenen tüm ROS konuları, servisleri ve parametreleri için bir isim alanı ön eki belirler. Birden fazla robotun olduğu simülasyonlarda çakışmayı önlemek için bu zorunludur.

Bu noktada, `gazebo_ros_control` ekosisteminin tam bir resmini görmek önemlidir. PID kazançları gibi kontrol parametrelerinin XML içinde nasıl ayarlandığına dair bir arayış, bu sistemin nasıl çalıştığına dair kritik bir gerçeği ortaya koyar: **PID kazançları XML'de ayarlanmaz.**

`gazebo_ros_control` ekosistemi üç parçalı bir sistemdir:

1. **URDF (`<transmission>`):** Hangi joint'in hangi *tür* arayüze (örn. `EffortJointInterface`) sahip olduğunu tanımlar.
2. **URDF (`<plugin>`):** Bu arayüzü Gazebo'da başlatan ve ROS'a bağlayan "sürücü" eklentisini (`libgazebo_ros_control.so`) yükler.
3. **YAML Dosyası (XML değil):** Bu arayüzün *nasıl* davranacağını tanımlar. Bu dosya, `roslaunch`

tarafından ROS parametre sunucusuna yüklenir ve hangi kontrolörün (örn.

`joint_state_controller`, `effort_controllers/JointPositionController`) yükleneceğini ve bu kontrolörün `p`, `i`, `d` kazançlarının ne olacağını belirtir.

Sadece XML dosyalarına bakan bir geliştirici, robotunun neden "sarkık" (floppy) olduğunu veya kontrol edilemediğini anlayamaz. Eksik olan parça, `roslaunch` ile yüklenen bu harici YAML yapılandırma dosyasıdır.

VI. Gelişmiş Düğüm: Animatörler (SDF)

Bu bölüm, dünyanızdaki statik modellerin ötesine geçen, senaryo tabanlı, animasyonlu varlıklara odaklanır.

A. Animasyonlu Model: `<actor>` Düğümü

- **Amacı:** İnsanlar, hayvanlar veya karmaşık hareket yolları izleyen dronlar gibi iskelet animasyonuna sahip varlıkları tanımlar. Bu varlıklar, önceden tanımlanmış bir yol (`<trajectory>`) üzerinde hareket ederken aynı zamanda bir iskelet animasyonunu (`<animation>`) oynatabilir.
- **Kod Örneği:**

```
<actor name="actor">
  <skin>
    <filename>walk.dae</filename>
  </skin>
  <animation name="walking">
    <filename>walk.dae</filename>
    <interpolate_x>true</interpolate_x>
  </animation>
  <script>
    <trajectory id="0" type="walking">
      <waypoint>
        <time>0</time>
        <pose>0 2 0 0 0 -1.57</pose>
      </waypoint>
      <waypoint>
        <time>2</time>
        <pose>0 -2 0 0 0 -1.57</pose>
      </waypoint>
      <waypoint>
        <time>7.5</time>
        <pose>0 2 0 0 0 -1.57</pose>
      </waypoint>
    </trajectory>
  </script>
</actor>
```

Alt Düğümlerin Analizi:

- `<skin>` : Varlığın görünümünü (3D model) tanımlar.
- `<animation>` : Varlığın iskelet animasyonunu (örn. yürüme döngüsü) tanımlar.
- `<script>` : Varlığın dünyadaki hareketini yönetir.
- `<trajectory>` : Varlığın izleyeceği yolu tanımlar. Buradaki `type="walking"` özniteliğinin, `<animation name="walking">` özniteliğiyle eşleştigiğine dikkat edin.
- `<waypoint>` : Yörünge üzerindeki `<time>` (zaman) ve `<pose>` (konum/duruş) anahtar karelerini tanımlar.

`<actor>` düğümü, aslında iki farklı animasyon türünü aynı anda senkronize eder. `<animation>` bloğu,

varlığın iskeletinin *yerinde* nasıl hareket edeceğini (örn. bacakların yürüme animasyonu) tanımlar.

`<script>` ve `<trajectory>` bloğu ise, varlığın *model kökünün* dünyada nasıl hareket edeceğini (örn. A noktasından B noktasına gitmek) tanımlar. `type` özneliği bu ikisini birbirine bağlar ve

`<interpolate_x>true</interpolate_x>` ayarı, Gazebo'nun modelin yörünge boyunca ilerleme hızına göre iskelet animasyonunun hızını otomatik olarak ayarlamasını sağlar.

VII. Özet Tablo: Temel Düğümler ve Kullanım Alanları

Bu rapor, Gazebo eğitimlerinde karşılaşılan birçok farklı XML ögesini incelemiştir. Bu ögelerin işlevselliği, büyük ölçüde içinde bulundukları bağlama (SDF dünyası mı, yoksa URDF robot tanımı mı?) bağlıdır.

Aşağıdaki tablo, bu temel düğümleri ve kullanım alanlarını özetleyerek bir "Rosetta Taşı" (çeviri anahtarı) görevi görmektedir.

Bu tablo, bir eğitimde karşılaşılan herhangi bir düğümün amacını ve bağlamını hızla teşhis etmek için bir araç olarak tasarlanmıştır. Örneğin, bir `<plugin>` etiketi görüldüğünde, bu tablodaki "Format" sütunu, kullanıcının bunun bir SDF dünyasında mı yoksa bir URDF dosyasında mı olduğunu kontrol etmesi gerektiğini belirtir. Benzer şekilde, `<joint>` (mekanizma) ve `<transmission>` (mekanizma için ROS kancası) arasındaki ayrımı netleştirir. Bu, Gazebo XML ekosisteminin tamamı için bir teşhis haritasıdır.

Gazebo XML Düğüm Referans Tablosu

Düğüm (Node)	Birincil Kullanım (Primary Use)	Format	Anahtar Alt Düğümler / Öznelikler
<code><physics></code>	Simülasyonun fizik motorunu ve zaman adımlarını küresel olarak ayarlar.	SDF	<code>type</code> , <code><max_step_size></code> , <code><real_time_update_rate></code> , <code><ode></code>
<code><plugin></code>	C++ kodu yükleyerek simülasyonu (Dünya/ GUI/Sensör/Model) genişletir.	SDF / URDF	<code>name</code> , <code>filename</code> , (parametreler)
<code><link></code>	Bir modelin (örn. robot) tek bir rijit (sert) gövdesini tanımlar.	SDF / URDF	<code><inertial></code> , <code><collision></code> , <code><visual></code>
<code><joint></code>	İki <code><link></code> ögesini birbirine bağlar ve aralarındaki hareketi kısıtlar.	SDF / URDF	<code>type</code> , <code><parent></code> , <code><child></code> , <code><axis></code>
<code><transmission></code>	Bir <code><joint></code> ögesini <code>gazebo_ros_control</code> eklentisine (ROS) kaydeder.	URDF	<code><hardwareInterface></code> , <code><joint></code> , <code><actuator></code>
<code><sensor></code>	Bir <code><link></code> içine yerleştirilir ve simülasyon dünyasından veri toplar.	SDF	<code>type</code> , <code>name</code> , <code><update_rate></code> , <code><noise></code> , <code><camera></code> , <code><ray></code> , <code><imu></code>
<code><gazebo></code>	URDF dosyalarına SDF'ye özgü özellikler (sensörler, eklentiler, fizik) ekler.	URDF	<code>reference</code> (bir linke eklerse), (içine <code><plugin></code> , <code><sensor></code> vb. alır)
<code><actor></code>	İskelet animasyonunu bir hareket yoluyla birleştiren animasyonlu varlık.	SDF	<code><skin></code> , <code><animation></code> , <code><script></code> , <code><trajectory></code>
<code><noise></code>	Bir <code><sensor></code> tanımı içine yerleştirilerek gerçekçi olmayan mükemmelliği bozar.	SDF	<code>type</code> , <code><mean></code> , <code><stddev></code> , <code><bias_mean></code>

VIII. Sonuç

Bu rapor, Gazebo simülasyon eğitimlerinde bulunan XML düğümlerine ilişkin bir taleple başlamış olsa da, analiz, bu düğümlerin Gazebo ekosisteminin temelini oluşturan daha karmaşık bir ikiliği (SDF ve URDF) ortaya çıkardığını göstermiştir. Sağlanan XML kod örnekleri, izole parçacıklar değil, fizik, kinematik, algılama ve kontrolü yöneten birbirine bağlı bir sistemin bileşenleridir.

Gazebo'da yetkinlik kazanmanın anahtarı, sadece bu XML öğelerinin sözdizimini ezberlemek değil, aynı zamanda içinde bulunulan formatı ve bu formatların nasıl etkileşime girdiğini (`<gazebo>` etiketi ve `gazebo_ros_control` gibi eklentiler aracılığıyla) anlamaktır. Özellikle ROS entegrasyonu söz konusu olduğunda, sistemin tam davranışının (örn. kontrolör kazançları), XML'in tamamen dışında, harici YAML yapılandırma dosyalarında tanımlandığı anlaşılmalıdır. Bu raporda, Gazebo'daki XML düğümlerinin nasıl çalışması gerektiği gösterilmiş oldu.

Özellikle ROS entegrasyonu söz konusu olduğunda, sistemin tam davranışının (örn. kontrolör kazançları), XML'in tamamen dışında, harici YAML yapılandırma dosyalarında tanımlandığı anlaşılmalıdır.